

Travaux Pratiques, séance 2

Problème de la diffusion (première partie)

Question 1. Implémentez l'algorithme CAST en *C*, avec la librairie MPI en utilisant l'arbre recouvrant donné par la structure de tas binaire. Le message envoyé sera composé de deux entier initialement générés aléatoirement par le processus d'identifiant 0. Chacun des processus devra afficher le message reçu. Exécutez votre programme avec 4, 5, 8, 10, et 16 processus.

La commande `MPI_Recv` peut être utilisée pour recevoir un message sans que l'identifiant de ce voisin soit précisé. Pour cela, il suffit de remplacer l'argument `origine_id` par `MPI_ANY_SOURCE`. Dans ce cas, un message sera reçu, quel que soit sa source. De même, cette commande peut recevoir un message quelle que soit sa balise en utilisant `MPI_ANY_TAG` à la place de l'argument `balise`.

Lorsqu'un message est reçu de cette façon, il est possible d'accéder tout de même à l'identifiant de l'entité de calcul qui l'a envoyé en utilisant la variable `status` de la manière suivante :

```
origine_id=status.MPI_SOURCE;
```

De même, la balise peut être obtenue de la manière suivante :

```
balise=status.MPI_TAG;
```

Question 2. Implémentez l'algorithme FLOOD en *C*, avec la librairie MPI. Vous supposerez que chaque entité de calcul est connectée à toutes les autres (ou, de manière équivalente, que le graphe du système distribué est complet, ce qui est l'hypothèse par défaut de MPI). Le message envoyé sera composé de deux entier initialement générés aléatoirement. Chacun des processus devra afficher le message reçu. Exécutez votre programme avec 10 processus.

Lors d'une réception, la taille du message à recevoir n'est pas forcément connu. Il est alors possible de recevoir la taille du message, avant de recevoir le message

lui-même, ce qui permet d'allouer la mémoire nécessaire de manière dynamique. Pour cela, il suffit d'utiliser la commande `MPI_Probe` :

```
MPI_Probe(origine_id, balise, MPI_COMM_WORLD, &status);
```

Il est là aussi possible de remplacer `origine_id` et `balise` par `MPI_ANY_SOURCE` et `MPI_ANY_TAG`. La taille du message à recevoir peut être alors obtenue par la commande

```
MPI_Get_count(&status, MPI_Type, &taille);
```

Question 3. Modifiez votre programme précédent pour que, la taille du message soit initialement généré au hasard entre 1 et 10. Le contenu du message sera lui aussi généré aléatoirement.

Question 4. Modifiez votre programme afin que chacune des entités de calcul renvoie un accusé de réception à l'entité de calcul de laquelle il a reçu le premier message. Cet accusé de réception sera l'entier 1. Qu'observez-vous ?

Question 5. Modifiez votre programme de façon que les entités de calcul qui ne reçoivent pas d'accusé de réception affichent le message « feuille », et que les autres entités de calcul affichent le message « parent de k feuilles », où k est le nombre d'accusés de réception qu'elles ont reçus d'entités de calcul qui ont affiché le message « feuille ».

Question 6. Écrivez un programme en *C* avec la librairie MPI qui se déroule de la manière suivante :

- Pour tout i , l'entité de calcul d'identifiant i génère d nombres $v_1, \dots, v_d$ distincts choisis dans $\{0, \dots, i-1\} \cup \{i+1, \dots, n-1\}$ où n est le nombre total d'entités de calcul du système distribué. Le nombre d sera défini avec une directive `#define`,
- Cette entité de calcul envoie un message aux entités de calcul d'identifiants $v_1, \dots, v_d$ pour leur dire qu'elle sera leur voisines,
- Cette entité de calcul stocke les identifiants de ses voisins dans un tableau `voisin`. Attention : une entité de calcul doit avoir au moins d voisins mais peut en avoir plus que d !

Question 7. Le programme de la question précédente définit un graphe G dont les sommets sont les entités de calcul du système distribué et dont les arêtes relient

deux entités voisines. Exprimez la valeur de d en fonction de n à partir de laquelle, ce graphe est nécessairement connexe. Exprimez la valeur de d en fonction de n en dessous de laquelle, ce graphe est nécessairement non-connexe.

Question 8. Répondez de nouveau aux questions 2 à 5 en supposant que l'algorithme FLOOD s'exécute dans le graphe G généré à la question 6 et non plus dans le graphe complet.